知乎 首发于 AlwaysBeta 切换模式 写文章

每周一个 Python 模块 | subprocess

程序员贝塔

60 人赞同了该文章

赞同 60

分享

更多技术文章，请关注：

yongxinz/tech-blog
github.com/yongxinz/tech-blog

同时，也欢迎关注我的微信公众号 **AlwaysBeta**，更多精彩内容等你来。

weixin.qq.com/r/jDgRCcr... (二维码自动识别)

subprocess 模块主要用于创建子进程，并连接它们的输入、输出和错误管道，获取它们的返回状态。通俗地说就是通过这个模块，你可以在 Python 的代码里执行操作系统级别的命令，比如 `ipconfig` 、 `du -sh` 等。

subprocess 模块替代了一些老的模块和函数，比如： `os.system` 、 `os.spawn*` 等。

subprocess 过去版本中的 `call()` ， `check_call()` 和 `check_output()` 已经被 `run()` 方法取代了。 `run()` 方法为 3.5 版本新增。

大多数情况下，推荐使用 `run()` 方法调用子进程，执行操作系统命令。在更高级的使用场景，你还可以使用 Popen 接口。其实 `run()` 方法在底层调用的就是 Popen 接口。

subprocess.run

subprocess.run(args, *, stdin=None, input=None, stdout=None, stderr=None, shell=False, timeout=None, check=False, encoding=None, errors=None)

功能：执行 args 参数所表示的命令，等待命令结束，并返回一个 CompletedProcess 类型对象。

注意，run() 方法返回的不是我们想要的执行结果或相关信息，而是一个 CompletedProcess 类型对象。

上面参数表里展示的只是一些常用的，真实情况还有很多。

args：表示要执行的命令，必须是一个字符串，字符串参数列表。

stdin、stdout 和 stderr：子进程的标准输入、输出和错误。其值可以是 `subprocess.PIPE` 、 `subprocess.DEVNULL` 、一个已经存在的文件描述符、已经打开的文件对象或者 None。

`subprocess.PIPE` 表示为子进程创建新的管道， `subprocess.DEVNULL` 表示使用 `os.devnull` 。默认使用的是 None，表示什么都不做。另外，stderr 可以合并到 stdout 里一起输出。

timeout：设置命令超时时间。如果命令执行时间超时，子进程将被杀死，并弹出 `TimeoutExpired` 异常。

check：如果该参数设置为 True，并且进程退出状态码不是 0，则弹出 `CalledProcessError` 异常。

encoding：如果指定了该参数，则 stdin、stdout 和 stderr 可以接收字符串数据，并以该编码方式编码。否则只接收 bytes 类型的数据。

▲ 赞同 60 ▼ 4 条评论 分享 喜欢 收藏 申请转载 ···

看下面的例子：

```
>>> subprocess.run(["ls", "-l"])  # 没有对输出进行捕获
CompletedProcess(args=['ls', '-l'], returncode=0)

>>> subprocess.run("exit 1", shell=True, check=True)
Traceback (most recent call last):
  ...
subprocess.CalledProcessError: Command 'exit 1' returned non-zero exit status 1

>>> subprocess.run(["ls", "-l", "/dev/null"], stdout=subprocess.PIPE)
CompletedProcess(args=['ls', '-l', '/dev/null'], returncode=0,
stdout=b'crw-rw-rw- 1 root root 1, 3 Jan 23 16:23 /dev/null\n')

>>> subprocess.run("python --version", stdout=subprocess.PIPE)
CompletedProcess(args='python --version', returncode=0, stdout=b'Python 3.6.1\r\n')

>>>s= subprocess.run("ipconfig", stdout=subprocess.PIPE)    # 捕获输出
>>>print(s.stdout.decode("GBK"))
```

subprocess.CompletedProcess

run() 方法的返回值，表示一个进程结束了。 `CompletedProcess` 类有下面这些属性：

- args 启动进程的参数，通常是个列表或字符串。
- returncode 进程结束状态返回码。0表示成功状态。
- stdout 获取子进程的 stdout。通常为 bytes 类型序列，None 表示没有捕获值。如果你在调用 run() 方法时，设置了参数 `stderr=subprocess.STDOUT` ，则错误信息会和 stdout 一起输出，此时 stderr 的值是 None。
- stderr 获取子进程的错误信息。通常为 bytes 类型序列，None 表示没有捕获值。
- check_returncode() 用于检查返回码。如果返回状态码不为零，弹出 `CalledProcessError` 异常。

subprocess.DEVNULL

一个特殊值，用于传递给 stdout、stdin 和 stderr 参数。表示使用 `os.devnull` 作为参数值。

subprocess.PIPE

管道，可传递给 stdout、stdin 和 stderr 参数。

subprocess.STDOUT

特殊值，可传递给 stderr 参数，表示 stdout 和 stderr 合并输出。

args 与 shell

args 参数可以接收一个类似 `'du -sh'` 的字符串，也可以传递一个类似 `['du', '-sh']` 的字符串分割列表。shell 参数默认为 False，设置为 True 的时候表示使用操作系统的 shell 执行命令。

下面我们来看一下两者的组合结果。

```
In [14]: subprocess.run('du -sh')
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
......
FileNotFoundError: [Errno 2] No such file or directory: 'du -sh'

In [15]: subprocess.run('du -sh', shell=True)
175M    .
```

可见，在 Linux 环境下，当 args 是个字符串时，必须指定 shell=True。成功执行后，返回一个 CompletedProcess 对象。

```
In [16]: subprocess.run(['du', '-sh'], shell=True)
.....大量的数据
4   ./文档
179100  .
Out[16]: CompletedProcess(args=['du', '-sh'], returncode=0)

In [17]: subprocess.run(['du', '-sh'])
175M    .
Out[17]: CompletedProcess(args=['du', '-sh'], returncode=0)
```

可见，当args是一个 `['du', '-sh']` 列表，并且 `shell=True` 的时候，参数被忽略了，只执行不带参数的 `du` 命令。

总结：Linux 中，当 args 是个字符串是，请设置 shell=True，当 args 是个列表的时候，shell 保持默认的 False。

获取执行结果

run() 方法返回的是一个 CompletedProcess 类型对象，不能直接获取我们通常想要的结果。要获取命令执行的结果或者信息，在调用 run() 方法的时候，请指定 stdout=subprocess.PIPE。

```
>>> ret = subprocess.run('dir', shell=True)
>>> ret
CompletedProcess(args='dir', returncode=0)


>>> ret = subprocess.run('dir', shell=True, stdout=subprocess.PIPE)
>>> ret
CompletedProcess(args='dir', returncode=0, stdout=b' \xc7\xfd\xb6\xaf\xc6\xf7 ......')

>>> ret.stdout
b' \xc7\xfd\xb6\xaf\xc6\xf7 C \xd6\xd0\xb5\xc4\xbe\xed\xca\xc7 ......'
```

从例子中我们可以看到，如果不设置 `stdout=subprocess.PIPE` ，那么在返回值 `CompletedProcess(args='dir', returncode=0)` 中不会包含 stdout 属性。反之，则会将结果以 bytes 类型保存在 ret.stdout 属性中。

交互式输入

并不是所有的操作系统命令都像‘dir’或者‘ipconfig’那样单纯地返回执行结果，还有很多像‘python’这种交互式的命令，你要输入点什么，然后它返回执行的结果。使用run()方法怎么向stdin里输入？

这样？

```
import subprocess

ret = subprocess.run("python", stdin=subprocess.PIPE, stdout=subprocess.PIPE,shell=Tru
ret.stdin = "print('haha')"     # 错误的用法
print(ret)
```

这样是不行的，ret作为一个 `CompletedProcess` 对象，根本没有stdin属性。那怎么办呢？前面说了，run()方法的stdin参数可以接收一个文件句柄。比如在一个 `1.txt` 文件中写入 `print('i like Python')` 。然后参考下面的使用方法：

```
import subprocess

fd = open("d:\\1.txt")
ret = subprocess.run("python", stdin=fd, stdout=subprocess.PIPE,shell=True)
print(ret.stdout)
fd.close()
```

这样做，虽然可以达到目的，但是很不方便，也不是以代码驱动的方式。这个时候，我们可以使用Popen类。

subprocess.Popen()

用法和参数与run()方法基本类同，但是它的返回值是一个Popen对象，而不是 `CompletedProcess` 对象。

```
>>> ret = subprocess.Popen("dir", shell=True)
>>> type(ret)
<class 'subprocess.Popen'>
>>> ret
<subprocess.Popen object at 0x0000000002B17668>
```

Popen对象的stdin、stdout和stderr是三个文件句柄，可以像文件那样进行读写操作。

```
>>>s = subprocess.Popen("ipconfig", stdout=subprocess.PIPE, shell=True)
>>>print(s.stdout.read().decode("GBK"))
```

要实现前面的‘python’命令功能，可以按下面的例子操作：

```
import subprocess

s = subprocess.Popen("python", stdout=subprocess.PIPE, stdin=subprocess.PIPE, shell=Tr
s.stdin.write(b"import os\n")
s.stdin.write(b"print(os.environ)")
s.stdin.close()

out = s.stdout.read().decode("GBK")
s.stdout.close()
print(out)
```

通过 `s.stdin.write()` 可以输入数据，而 `s.stdout.read()` 则能输出数据。

相关文档：

pymotw.com/3/subprocess...

liujiangblog.com/course...

manongjc.com/article/24...

编辑于 2020-01-19 23:05

Python　Python 开发　Python教程

写下你的评论...

4 条评论　默认　最新

梦游的树
请问在stdin=subprocess.PIPE方式下，如果是调用ffmpeg合并视频和音频，有没有办法多管道输入？
2023-07-12 回复 喜欢

瞌睡虫
催更催更
2022-11-26 回复 喜欢

Chuchucon
run和Popen都是等待至子进程结束吗？
2022-09-23 回复 喜欢

shayu0789
run是，Popen不是
2023-03-22 回复 2

写下你的评论...

文章被以下专栏收录

AlwaysBeta
专注后端开发，微信公众号：程序员贝塔，欢迎关注。

推荐阅读

Python的51个“秘密”被曝光 |GitHub超2万星
七月在线 ... 发表于从零学AI

python进阶到高阶大全（强烈推荐）
python进阶到高阶大全（强烈推荐）_Paulzhao6518的博客-CSDN博客_python进阶关键字is 和 == 的区别 a = 'hello world' b = 'hello world' a == b #返回True ...
李先生

python官方文档教程0-3章学习笔记
学习教程——https://docs.python.org/zh-cn/3/tutorial/index.html为什么要从零开始跟着文档学习python？ 因为我的python水平太烂了，一个简单的自动化报表生成，花了20多...
孙墨

Python从菜鸟到高手（1）：初识Python
蒙娜丽宁